```
  } else {
    return "yellow"
  }
}

interface SquareProps {
  status: string
  className?: string
}
const buttonClass = "p-4 m-2 my-10 border-black border-4 text-lg"

const ButtonSquare = styled.span.attrs({
  className: buttonClass,
})<SquareProps>`
  background-color: ${(props) => stateColor(props.status)};
  transition: all 1s ease-in-out;

  &:hover {
    background-color: ${(props) =>
      props.status === "unsold" ? "lightblue" : stateColor(props.status)};
  }
`


interface SeatProps {
  seatNumber: number
  rowNumber: number
}

export const Seat = ({
  seatNumber,
  rowNumber,
}: SeatProps): React.ReactElement => {
  const context = React.useContext<IsVenueContext>(VenueContext)

  const seatMatch = (ticketList: TicketData[], exact = false): boolean => {
    for (const heldTicket of ticketList) {
      const rowMatch = heldTicket.row == rowNumber
      const seatDiff = heldTicket.number - seatNumber
      const diff = exact ? 1 : context.state.ticketsToBuyCount
      const seatMatch = seatDiff >= 0 && seatDiff < diff
      if (rowMatch && seatMatch) {
        return true
      }
    }
    return false
```